Limpieza de datos de un proyecto de RFM ((Recencia, Frecuencia, Valor Monetario). Proyecto de portafolio de Analista de Datos Jr

Librerías que se usaran en este proyecto

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
import json
import math
#Para hacer limpieza de errores tipograficos a gran escala
import unicodedata
import re
#Para el calculo de los productos que se venden en pareja
from itertools import combinations
from collections import Counter
```

Estas son las numerables librerías que se estaría usando en este proyecto las cuales serán aclaradas en otra presentación, pero por ahora solo se estarían usando:

- **Pandas:** Utilizada para la manipulación y análisis de datos. Crea y gestiona DataFrames, carga archivos (CSV, Excel), realiza operaciones de limpieza, filtrado, agrupación y unión de datos.
- **Matplotlib:** Es la librería fundamental para la creación de gráficos estáticos, aunque en este código, muchas visualizaciones se delegan a Seaborn, Matplotlib se usa implícitamente por Seaborn y para la configuración general de las figuras y subgráficos.
- **Seaborn:** Se utiliza para crear visualizaciones estadísticas más atractivas y complejas a partir de los DataFrames de Pandas, como boxplots, histogramas, barplots, lineplots y heatmaps, para entender distribuciones y patrones de datos.
- **JSON**: Se emplea para manejar datos en formato JSON, específicamente para guardar de forma segura las credenciales de la API de Kaggle en un archivo.
- **Unicodedata**: Se usa en la función de limpieza de texto para normalizar las cadenas de caracteres y eliminar acentos, estandarizando las descripciones de los productos.
- **re** (Regular Expressions): Esta librería de expresiones regulares se utiliza también en la función de limpieza de texto para eliminar caracteres especiales y estandarizar múltiples espacios en blanco, contribuyendo a la homogeneización de las descripciones.
- **os**: Permite interactuar con el sistema operativo, como crear directorios (`os.makedirs`) y gestionar rutas de archivos (`os.path.join`), esencial para la configuración inicial de Kaggle y el almacenamiento de resultados en Google Drive más adelante.

Para esta primera parte del proyecto de análisis de RFM

Importación del dataset desde kaggle

```
#Configuraciones de credenciales
data = {"username": "********", "key": "********************************"}
os.makedirs("/root/.kaggle", exist_ok=True)
with open("/root/.kaggle/kaggle.json", "w") as f:
    json.dump(data, f)
os.chmod("/root/.kaggle/kaggle.json", 600)

#Ruta oficial
print("Descarga del dataset fue realizada")
!kaggle datasets download -d mathchi/online-retail-ii-data-set-from-ml-
repository

#Descomprimiendo los archivos
!unzip -o online-retail-ii-data-set-from-ml-repository.zip

#Listar para confirmar
print("\nArchivos descargados:")
!ls -lh
```

Este código es como un pequeño asistente que te ayuda a descargar datos de ventas de una tienda en línea desde la página web Kaggle y prepararlos para que se pueda usar en esta libreta de trabajo. En la configuración de credenciales tenemos como la tarjeta de acceso para miembros. Seguido tenemos un mensaje de control

`print("Descarga del dataset fue realizada")`

Seguido de la línea más importante para descargar donde decimos cual conjunto de datos queremos descargar con el comando `!kaggle datasets download -d mathchi/` seguido del nombre del archivo que es `online-retail-ii-data-set-from-ml-repository`

Pero este archivo viene comprimido para su fácil descarga pero necesitamos descomprimirlo, esto lo hacemos en la siguiente línea usando el comando `!unzip` y sacamos todos los archivos pero también usamos este otro comando, El `-o` significa que si ya existe algún archivo con el mismo nombre, lo sobrescribirá sin preguntar.

Y por ultimo para esta celda tenemos la utilización de los comandos `!ls -lh` es como pedirle al asistente: "Muéstrame una lista de todos los archivos que hay en esta carpeta y dame los detalles (como su tamaño) de forma que sea fácil de leer". Así puedes confirmar que los archivos del dataset están presentes y listos para ser usados. Donde tenemos:
`'Year 2009-2010.csv'`

`'Year 2010-2011.csv'`

```
#Cargar los archivos Excel
#Este archivo tiene 2 hojas: 'Year 2009-2010' y 'Year 2010-2011'
df_2009 = pd.read_excel('online_retail_II.xlsx', sheet_name = 'Year 2009-
2010')
df_2010 = pd.read_excel('online_retail_II.xlsx', sheet_name = 'Year 2010-
2011')

#Uniendo ambos años
df_full = pd.concat([df_2009, df_2010], ignore_index = True)

#Modificando el formato para su presentacion
rows, cols = df_full.shape
formatted_rows = f"{rows:,}".replace(",", ".")
print(f"El dataframe df_full tiene {formatted_rows} filas y {cols} columnas")
```

Esta parte del código se encarga de tomar los archivos de Excel que acabamos de descomprimir, leer su contenido y juntarlo todo en una sola tabla grande para que podamos analizarla cómodamente. También nos dice el tamaño final de esa tabla.

Pero para explicarlo por partes tenemos el archivo online_retail_II.xlsx que es un archivo en Excel que contiene 2 libros los cuales son los ya mencionados, aquí se lee el primer libro “Year 2009-2010” con la función `pd.read_excel()` y se guarda temporalmente en un dataframe llamado df_2009 y asi con el otro libro “Year 2010-2011” y se crea el dataframe df_2010

Pero ahora tenemos 2 dataframe y como necesitamos una sola tabla para una mejor manipulación las unimos usando la función `pd.concat()`, pero como los datos tienen diferentes numeración en los índice usamos este parámetro `ignore_index = True` el cual es para que la tabla combinada tenga una numeración de filas nueva y continua, en lugar de mantener las numeraciones originales de las tablas separadas. Ahora si tenemos un dataframe completo el cual llamaremos df_full

Pero ahora que lo tenemos, se quiere de que esta compuesto y para esto usamos esta línea de código:

`rows, cols = df_full.shape`
Aquí usamos el commando `.shape` esto es como preguntarle a la tabla: "¿Cuántas filas y cuántas columnas tienes?". El código toma esa información y la muestra en un mensaje fácil de leer, por ejemplo: "El dataframe df_full tiene 1.067.371 filas y 8 columnas". Esto da una idea rápida del volumen de datos con el que se va a trabajar.

Pero como se quiere que se tenga separación entre miles entonces usamos esta línea para agregar un “.”como separador de miles

`formatted_rows = f"{rows:,}".replace(",", ".")`
Aquí estamos diciendo en el valor de las filas el valor colócale un “.” Como separador de miles para su mejor lectura pero como por defecto Python coloca “,” por eso se usa el método `replace()`

```
df_full.head(5)
```

Ahora fino sabemos cuantas columnas y filas se compone nuestro dataframe pero no conocemos su interior, como esta conformado, cual formato se usa en las fechas ni como se llaman las columnas entonces para esto imprimimos el dataframe pero como no queremos toda la tabla solo algunas filas usamos el método `.head()` que por defecto devuelve las primeras 5 filas que es lo suficiente para entender a que nos enfrentamos

```
df_full.info()
```

Ahora usando la función `.info()` en la siguiente celda tendríamos un resumen conciso del dataframe, Proporciona información esencial sobre la estructura de los datos, incluyendo el número de filas/columnas, tipos de datos (dtypes), valores no nulos y el uso de memoria, por ejemplo tenemos este resultado

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1067371 entries, 0 to 1067370
Data columns (total 8 columns):
 #   Column       Non-Null Count    Dtype
---  ------       --------------    -----
 0   Invoice      1067371 non-null  object
 1   StockCode    1067371 non-null  object
 2   Description  1062989 non-null  object
 3   Quantity     1067371 non-null  int64
 4   InvoiceDate  1067371 non-null  datetime64[ns]
 5   Price        1067371 non-null  float64
 6   Customer ID  824364 non-null   float64
 7   Country      1067371 non-null  object
dtypes: datetime64[ns](1), float64(2), int64(1), object(4)
memory usage: 65.1+ MB
```

El cual lo desglosamos así:

1. `<class 'pandas.core.frame.DataFrame'>`

Dice que lo que se está viendo es un objeto DataFrame de la librería pandas. En términos sencillos, es tu tabla de datos principal con filas y columnas.

2. `RangeIndex: 1067371 entries, 0 to 1067370`

Indica que tu tabla tiene 1,067,371 filas (o "entradas"). Las filas están numeradas del 0 al 1,067,370. Esto da una idea del volumen total de transacciones o registros que se tiene.

3. `Data columns (total 8 columns):`

La tabla tiene un total de 8 columnas. Cada columna representa una característica o tipo de información sobre cada transacción (como la factura, el producto, la cantidad, etc.).

4. `# Column Non-Null Count Dtype:`

Esta es la parte más crucial para entender la calidad de tus datos. Te muestra una lista de cada columna y, para cada una:

- `Non-Null Count` (Conteo de valores no nulos): Dice cuántos valores válidos (que no están vacíos o 'NaN') tiene esa columna. Si este número es menor que el RangeIndex total (1,067,371), significa que hay valores faltantes (nulos) en esa columna.
- `Dtype` (Tipo de dato): Indica el tipo de información que guarda la columna (por ejemplo, números enteros, números con decimales, texto, fechas, etc.). Es importante que los tipos de datos sean correctos para poder hacer operaciones matemáticas o de filtrado.

Análisis por columna:

- `Invoice, StockCode, Quantity, InvoiceDate, Price, Country`: Todas estas columnas tienen 1.067.371 non-null, lo que significa que no tienen valores faltantes y están completas. Sus tipos de datos (object para texto, int64 para enteros, datetime64[ns] para fechas y float64 para números decimales
- `Description` (Descripción del producto): Tiene 1.062.989 non-null. Esto significa que faltan 4.382 descripciones (1.067.371 – 1.062.989).
- `Customer ID` (ID del cliente): Tiene 824364 non-null. Esta es una columna con muchos valores faltantes. Hay 243.007 transacciones (1.067.371 – 824.364) que no tienen un ID de cliente asociado

5. `dtypes: datetime64[ns](1), float64(2), int64(1), object(4):`

Es un resumen de cuántas columnas hay de cada tipo de dato en tu tabla.

6. `memory usage: 65.1+ MB:`

Dice cuánta memoria RAM está utilizando la tabla de datos. Es útil para saber si el dataset es muy grande y podría ralentizar tu computadora o las operaciones.

```
df_full.isnull().sum()
```

Esta celda no es importante debido que ya tenemos mucha información con el uso de `.info()`, pero aquí tenemos un recuentro de los valores nulos, pero nos dice que los valores faltantes no son nulos en realidad son valores NaN

```
pd.options.display.float_format = '{:,.2f}'.format
df_full.describe()
```

`pd.options.display.float_format = '{:,.2f}'.format`

Le dice a pandas que, de ahora en adelante, cuando muestre números con decimales en sus tablas (DataFrame):

1. Les ponga un separador de miles (coma `,`).
2. Los muestre solo con dos decimales `(.2f)`.

Esto Hace que los números grandes y con decimales sean más fáciles de leer, como si estuvieras viendo una tabla de contabilidad.

Luego en la siguiente línea con `.describe()` tenemos una información más amplia del dataframe donde nos arroja esta información de solo las columnas métricas (columnas numéricas) las cuales son "Quantity, Price, Customer ID" y con fecha "InvoiceDate" para este ejemplo.

	Quantity	InvoiceDate	Price	Customer ID
count	1,067,371.00	1067371	1,067,371.00	824,364.00
mean	9.94	2011-01-02 21:13:55.394028544	4.65	15,324.64
min	-80,995.00	2009-12-01 07:45:00	-53,594.36	12,346.00
25%	1.00	2010-07-09 09:46:00	1.25	13,975.00
50%	3.00	2010-12-07 15:28:00	2.10	15,255.00
75%	10.00	2011-07-22 10:23:00	4.15	16,797.00
max	80,995.00	2011-12-09 12:50:00	38,970.00	18,287.00
std	172.71	NaN	123.55	1,697.46

Donde tenemos las filas:

- `count:` (Conteo) El número total de valores válidos (no vacíos) en cada columna.
- `mean`: (Promedio) El valor promedio de los datos en esa columna.
- `Min:` (Minimo) El valor más pequeño encontrado en esa columna.
- `25%`, `50%` (Mediana), `75%` (Cuartil) Estos son los percentiles. 25% es el valor por debajo del cual está el 25% de los datos; 50% (la mediana) es el punto medio (la mitad de los datos están por debajo y la otra mitad por encima); 75% es el valor por debajo del cual está el 75% de los datos.
- `Max` (Maximo): El valor más grande encontrado en esa columna.

- `Std` (Desviación Estándar) Mide cuánto se dispersan los datos con respecto al promedio. Una std alta indica que los datos están muy extendidos; una baja indica que están agrupados cerca del promedio

En general podemos decir que:

- Calidad de Datos: La columna Customer ID tiene valores faltantes significativos, y Quantity y Price tienen valores negativos que deben ser tratados con cuidado.
- Comportamiento de Compra: La mayoría de las transacciones son de pocas unidades y precios bajos, pero existen "outliers" (valores atípicos) de compras muy grandes (en cantidad y/o precio). Estos son valiosos y no deben eliminarse sin un análisis cuidadoso.
- Fechas: El rango de fechas de tu dataset es desde diciembre de 2009 hasta diciembre de 2011.

```
cols_num = ['Quantity', 'Price']

fig, axes = plt.subplots(1, 2, figsize=(15, 4))
fig.subplots_adjust(hspace=0.5)

red_outlier = dict(markerfacecolor='red', marker='o')

for i, col in enumerate(cols_num):
  sns.boxplot(data=df_full, x=col, ax=axes[i], flierprops=red_outlier)
  axes[i].set_title(col)
plt.show()
```

Aquí solo se quiere graficar para ver los valores outliers de una forma visual y como los mismos solo están en las columnas “'Quantity” y “Price”, creamos una lista con los mismos guardados en la variable `cols_num`, ahora continuamos con la construcción de las graficas donde

Ahora continuando, pero en la misma celda tenemos la gráfica para la visualización de estas columnas donde tenemos

```
fig, axes = plt.subplots(1, 2, figsize=(15, 4))
```

- `plt.subplots()` es una función de matplotlib que crea una figura (el lienzo completo del gráfico) y un conjunto de subgráficos o ejes.
- 1, 2 significa que queremos una cuadrícula de 1 fila y 2 columnas para nuestros gráficos.

- `figsize=(15, 4)` establece el tamaño de la figura en 15 pulgadas de ancho y 4 pulgadas de alto.
- El resultado se asigna a `fig` (la figura completa) y `axes` (un array de objetos de ejes, donde `axes[0]` sería el primer subgráfico y `axes[1]` el segundo).

2. `fig.subplots_adjust(hspace=0.5)`
   - `fig.subplots_adjust()` se usa para ajustar el espaciado entre los subgráficos.
   - `hspace=0.5` controla el espacio horizontal entre los subgráficos, lo que se usa cuando se tiene varias filas de gráficos, que este caso.
3. `red_outlier = dict(markerfacecolor='red', marker='o')`
   - Aquí se crea un diccionario llamado `red_outlier`. Este diccionario define propiedades visuales para los valores atípicos en los boxplots.
   - `markerfacecolor='red'` indica que el color de relleno del marcador del outlier será rojo.
   - `marker='o'` indica que el marcador para los outliers será un círculo.
4. `for i, col in enumerate(cols_num):`
   - `cols_num` es una lista que contiene los nombres de las columnas que se quiere analizar, en este caso las columnas `['Quantity', 'Price']`.
   - Por qué el bucle: El bucle se usa para evitar escribir el mismo código dos veces (o más) para cada columna. En lugar de copiar y pegar el código para `Quantity` y luego para `Price`, el bucle lo hace automáticamente por cada elemento en `cols_num`.
     - En la primera iteración, i será 0 y col será `'Quantity'`.
     - En la segunda iteración, i será 1 y col será `'Price'`.
   - Así tenemos dos graficas con poco código que es uno de los principios del Zen de Python
5. `sns.boxplot(data=df_full, x=col, ax=axes[i], color='blue', flierprops=red_outlier)`
   - `sns.boxplot()` es la función de seaborn para crear un diagrama de caja.
   - `data=df_full`: Indica que los datos provienen del DataFrame df_full.
   - `x=col`: Aquí es donde `col` del bucle es crucial. En cada iteración, `col` toma el nombre de la columna actual (`'Quantity'` o `'Price'`), y el boxplot se genera para esa columna.
   - `ax=axes[i]`: Esto asigna cada boxplot al subgráfico correcto (`axes[0]` para `Quantity` y `axes[1]` para `Price`).
   - `flierprops=red_outlier`: Aplica las propiedades de estilo que definimos anteriormente (`red_outlier`) a los outliers en el boxplot, haciéndolos visibles en rojo.
6. `axes[i].set_title(col)`
   - `axes[i].set_title()` establece el título de cada subgráfico.
   - A usa `col` en cada iteración la gráfica tomará el nombre de la columna que se estará iterando en esa ronda
7. `plt.show()` aquí solo imprimimos o mostramos los gráficos

```
col_cat = ['StockCode', 'Description', 'Country']
#Valores unicos
for col in col_cat:
  print(f"Column \033[1m{col}\033[0m {df_full[col].nunique()}")
```

Con esta lista se busca ver lo que se tiene en las otras columnas faltantes, para luego en el bucle iterar cada una de las mismas y tener un recuento de los valores únicos en las columnas con el uso del método `.nunique()`, ejemplo aquí se puede ver que hay 5698 productos distintos en la columna description.

Aquí podemos ver el uso del código ANSI en la línea `print` con el uso de `\033[1m{col}\033[0m` aquí solo se quiere darles una mejor presentación a los resultados colocando en negritas el nombre de las columnas

```
#Funcion para aplicar normalizacion al texto en la columna description
def order_description (texto):
  if pd.isna(texto): return "nulo"
  texto = str(texto).lower().strip() #todo a minusculas y quitar espacios extremos
  texto = "".join(c for c in unicodedata.normalize('NFD', texto) if
unicodedata.category(c) != 'Mn') #Eliminar acentos
  texto = re.sub(r'[^a-zA-Z0-9\s]', '', texto) #Eliminar caracteres especiales
  texto = re.sub(r'\s+', ' ', texto) #Eliminar espacios en blanco adicional
  palabras = sorted(texto.split()) #organizar palabras
  return " ".join(palabras)
```

Ahora queremos normalizar y eliminar duplicados en la columna description debido que podemos tener un mismo producto descrito de diferentes manera por ejemplo:

`blue door french metal sign == FRENCH BLUE METAL DOOR SIGN`

`font number tile vintage == NUMBER TILE VINTAGE FONT`

Aquí temenos 2 productos descritos en 2 entradas de manera diferentes, es decir 2 productos escritos de diferentes maneras, esto puede ser error al cargar la data del almacén, para esto necesitamos normalizar el texto en cada una de las filas con la función `def order_description (texto):` donde “texto” es la entrada o la descripción en la columna description, luego tenemos el manejo de nulos con el condicional `if,` después a la entrada o variable texto se le va aplicando:

- Normalización básica: Convierte el texto a minúsculas y elimina los espacios en blanco al principio y al final de la cadena.
- Eliminación de acentos: Remueve los acentos y otros diacríticos de los caracteres.
- Filtro de caracteres: Elimina cualquier carácter que no sea alfanumérico o un espacio, dejando solo letras, números y espacios.
- Espacios consistentes: Reemplaza múltiples espacios consecutivos por un único espacio.
- Ordenamiento de palabras: Divide el texto en palabras, las ordena alfabéticamente y luego las vuelve a unir.

Al final tenemos este retorno

`return " ".join(palabras)`

Que realiza lo siguiente:

`palabras`: En este punto de la función, palabras es una lista de cadenas de texto (las palabras de la descripción original), que ya han sido procesadas (minúsculas, sin acentos, sin caracteres especiales, y ordenadas alfabéticamente).

El método `.join()`: Este es un método de las cadenas de texto en Python. Su función es concatenar todos los elementos de un iterable (en este caso, la lista palabras) en una sola cadena. El elemento sobre el que se llama join() (aquí " ") actúa como el separador que se intercala entre cada elemento de la lista.

Así, `" ".join(palabras)` toma todas las palabras de la lista palabras y las une en una única cadena, colocando un espacio entre cada palabra. El resultado final es una descripción de producto limpia y normalizada, con sus palabras ordenadas alfabéticamente y separadas por un solo espacio.

```
#Creando una columna temporal
df_full['description_temp'] = df_full['Description'].apply(order_description)

#Buscando parentesco
errors_desc = df_full.groupby('description_temp')['Description'].unique()

#Filtrando las coincidencia con mas de una version
possible_dup = errors_desc[errors_desc.str.len() > 1]

print(f"Se encontraron {len(possible_dup)} posibles duplicados")
print("Ejemplo:")
print(possible_dup.head(10))
```

Continuando con la normalización del texto en la columna description seguimos con esta otra celda donde creamos una nueva columna temporal llamada `'description_temp'` a partir de la columna original `description` y donde aplicamos la función anteriormente mencionada `order_description (texto)`, en otra línea buscamos parentesco aplicando el método `.groupby()` y se aplica `unique()` con el fin de recolectar todos los valores únicos de la columna `description` originales asociados al grupo recién hecho . Esto para identificar si una

`description_temp` (la versión limpia) se corresponde con múltiples descripciones (versiones originales con errores o variaciones), que es precisamente lo que se busca para detectar 'parentesco' o posibles duplicados.

```
#Eliminacion de texto mal escritos en description
#Mapeo de la columna description
mapping_correction = {}

for clean, original in possible_dup.items():
  official_version = original[0]
  for error in original[1:]:
    mapping_correction[error] = official_version

print(f"Se encontraron y se corrigieron {len(mapping_correction)} variantes")
```

En esta siguiente celda creamos un diccionario vacío como inicialización que lo iremos llenando con las iteraciones de los bucles más adelante

1. Donde tenemos la Iteración: Recorre los elementos de `possible_dup`. Cada elemento de `possible_dup` es una tupla donde `clean` es la versión normalizada de una descripción y `original` es una lista de todas las versiones originales (incluyendo la "oficial" y sus variantes) que corresponden a esa descripción normalizada.
2. Versión oficial: Para cada `clean` (descripción normalizada), se toma el primer elemento de la lista original (`original[0]`) como la `official_version` (la forma correcta de la descripción).
3. Mapeo de errores: Luego, se itera sobre el resto de los elementos de la lista `original` (`original[1:]`), que son las variantes o errores tipográficos. Para cada uno de estos `error`, lo añade al diccionario `mapping_correction` mapeándolo a la `official_version`.
4. Conteo: Finalmente, se imprime cuántas variantes únicas fueron identificadas y mapeadas para su corrección solo de manera de visualización.

Es decir, esta celda construye un "mapa" donde cada descripción mal escrita o variante apunta a su versión correcta y estandarizada, facilitando la posterior sustitución en el DataFrame.

```
#Sustitucion de la columna description por una nueva ya normalizada y sin
errores de tipado
df_full['Description'] = df_full['Description'].replace(mapping_correction)

print(df_full['Description'].unique().shape[0])
```

Aquí sustituimos los valores de “Description” por los valores que se encuentran `mapping_correction` que se hizo anteriormente con el texto ya corregidos e imprimimos nuevamente para ver cuantos valores tenemos en la columna “Description” donde se modificaron 215 descripciones de producto Se tiene que aclarar que el procedimiento anterior no es tan necesario el cálculo de RFM debido que se mantiene el número de transacciones pero si es bueno tener esta columna normalizada para el caculo de algún top 10 de productos mas vendidos

```
df_full.drop(columns=['description_temp'], inplace=True)
```

En esta celda simplemente se elimina la columna `description_temp` debido que ya cumplió con su objetivo y se aplica el parámetro `inplace=True` para que los cambios se apliquen en el dataframe (df_full) y no en otra copia

```
#Chequeo de la columna country en busca de errores
country_unique = df_full['Country'].unique().astype(str)
country_unique.sort()
print(country_unique)
```

En esta celda solo se busca chequear la columna country buscar errores y/o datos no acordes a la descripción de la columna. Aquí obtenemos primeramente los valores únicos con el método `unique()`, además se usa `.astype(str)` para para convertir el tipo de datos de la columna del dataframe a una cadena de caracteres y posteriormente en la siguiente línea se ordenan con el método `.sort()` para así facilitar la lectura

```
#Se filtra 'Unspecified'
country_unique = country_unique[country_unique != 'Unspecified']
print(f"Total de paises: {len(country_unique)}")
```

En el resultado de lo que devolvió la celda anterior se observa la denominación `'Unspecified'` es decir hay países que se le colocaron la denominación “Sin especificar”, para el análisis de RFM esto no genera ningún problema debido que, si existe esta transacción solo que no esta claro donde fue originada, y en esta celda solo se esta comprobando el valor real de países únicos que en este caso la tienda trabaja en 42 países

La lógica de esta celda es:

La línea `country_unique = country_unique[country_unique != 'Unspecified']` lo que hace es filtrar el arreglo `country_unique`.

1. `country_unique != 'Unspecified'`: Esta parte crea una serie de valores booleanos (True o False) del mismo tamaño que country_unique. Será True para cada país que no sea igual a `'Unspecified'`, y False para aquellos que sí lo sean.
2. `country_unique[...]`: Al usar esta serie booleana como un índice dentro de country_unique, se seleccionan solo los elementos donde la condición es True. Es decir, se crea un nuevo arreglo country_unique que contiene todos los países que son distintos de 'Unspecified'.

Así que, resumiendo != indica 'distinto de' o 'no igual a'. La celda elimina el valor 'Unspecified' del listado de países y luego imprime la cantidad de países restantes

```
#Se tiene nombre en los países como EIRE que se refiere a Iraland, USA para
United States y otro
#Se sustituyen para su mejor lectura
country_mapping = {
    'EIRE': 'Ireland',
    'USA': 'United States',
    'RSA': 'South Africa'
}
```

Se crea una variable que almacena un diccionario con los nombres de los países por ejemplo EIRE, USA, RSA en la clave y se sustituye por lo valores Ireland, United States y South Africa respectivamente solo para mejorar la lectura

```
df_full['Country'] = df_full['Country'].replace(country_mapping)
```

Ahora en esta celda solo se sustituye los valores de la columna Country por los modificados de la variable `country_mapping` usando el método `.replace()`

```
#Se Contabiliza cuantas transacciones tienen la definicion de paises
'Unspecified'
total_unspecified = len(df_full[df_full['Country'] == 'Unspecified'])
print(f"Total de registros con paises 'Unspecified' en la columna 'country':
{total_unspecified}")
```

Esta celda solo es para verificación cuantas transacciones tienen en la columna country la denominación `Unspecified` usando la función `len ()` el cual arrojo la cifra de 756 piases, estas son transacciones verdaderas, pero sin una ubicación geográfica.

```
df_full.columns = df_full.columns.str.lower().str.strip().str.replace(' ',
'_')
```

En esta celda solo se está normalizando los títulos de las columnas para que este dataframe sea usado fácilmente en consultas de SQL donde se elimina las mayúsculas, los espacios en blanco como la columna Customer ID ahora se pasaría a llamar customer_id, esta celda no tiene nada que ver con calculo solo con limpieza

```
df_full[df_full['quantity'] > 4000].head()
df_full[df_full['price'] > 3000].head()
```

Ahora en estas 2 líneas que están en celdas separadas debido que solo es para visualizar el dataframe se está buscando de manera visual si los outliers que se observaron en las gráficas anteriores cumplen con las características de una transacción verdadera. Se usa el método `.head()` debido que no se quiere imprimir todo el dataframe solo apenas las primeras 5 líneas, aquí se observa que si cumplen con todo, hay una customer_id, no hay datos a lazar que indicaran que fue error de transcripción

```
#Calulo de df_full_to_geograpy
#Obteniendo todas las columnas
all_columns = df_full.columns.tolist()

#Excluyendo customer_id
check_for_nulls = [col for col in all_columns if col not in ['customer_id']]

#Aplicando dropna al resto de columnas
df_full_to_geograpy = df_full.dropna(subset=check_for_nulls).copy()
```

Ya con la limpieza (menos la eliminación de valores NaN) se procede con la creación del primer dataframe derivado de df_full(original), este busca tener las transacciones tomando en cuenta los países debido que se está omitiendo la gran cantidad de valores NaN en la columna customer_id. Aquí se copia todos los valores de todos los nombres de la columnas del dataframe, aquí se usa `tolist()` para convertir los elementos de objeto Index creado con el uso de `.columns` en una lista de Python

Ahora seguido de la línea `check_for_nulls = [col for col in all_columns if col not in ['customer_id']]` donde con esta lista de compresión se crea una lista con todo los nombre de las columnas excepto customer_id

Seguido de la siguiente línea `df_full_to_geograpy = df_full.dropna(subset=check_for_nulls).copy()` se borra todos los valores NaN excepto en la columna customer_id y se crea el dataframe `df_full_to_geograpy` con la finalidad de conservar el casi 22% que se hubiera perdido si se aplicara directamente `.dropna`

Si solo fuera para el cálculo de RFM estaría bien aplicar `.dropna` pero como se desea realizar otros cálculos y estas filas o transacciones están completas solo estaría faltando un ID a ese cliente pero si existen la transacción, se pudiera sustituir los valores NaN en customer_id usando el método `rename()` con otro valor o agregándole un id (manteniendo las normas de manejo de datos faltante) pero no es relevante para este proyecto

```
df_full.dropna(inplace=True)
```

Ahora si se procede a eliminar todos los valores NaN del dataframe df_full, para proceder con el análisis de RFM, este procedimiento no se realizo antes como ya se había mencionado para no perder el 22% de la data (la que tenía valores NaN en customer_id), y se quería realizar las diferentes limpiezas antes de derivar un dataframe primero y no tener que reusar código una vez mas solo para el dataframe `df_full_to_geograpy` si este se hubiera derivado al principio de la limpieza, por eso se dejó el uso de `.dropna` de ultimo

```
#Se pasa la columna Customer ID a entero para eliminar el .00
df_full['customer_id'] = df_full['customer_id'].astype(int)
```

Este procedimiento es solo de presentación debido que los valores de la columna customer_id presentaban valores decimales (.00) y aquí se elimina esos 2 ceros al final pasándolos a entero aplicando `.astype(int)`, esta celda no se desarrolló antes debido que hubiera dado error porque customer_id tenía valores NaN

```
df_full.info()
```

Aquí se vuelve a verificar de manera visual si se ejecutó con éxito la limpieza donde ahora tenemos 824.364 valores de 1.067.371 que tenía anteriormente, ahora si este dataframe esta listo para su uso en el análisis de RFM sin los valores NaN pero antes de esto verificaremos los valores negativos que se observaron anteriormente en la columna “Quantity” en la siguientes 2 celdas

```
negative_quantity = df_full[df_full['quantity'] < 0]
display(negative_quantity.head(10))
negative_quantity.shape[0]
```

En esta primera celda verificamos de manera visual la tabla solo con los valores negativos o menores a cero, de la columna quantity, y se imprime o devuelve apenas las primeras 10 filas con el uso de `head(10)`, en la siguiente línea `negative_quantity.shape[0]` se busca que se devuelva solo el valor de las filas con valores negativos con el uso del atributo `.shape[]`, 0 para ver solo las filas si se deseara solo las columnas seria 1 y sin shape pandas imprime ambos valores por defecto.

Nota: Se observa que en la columna "invoice", se tiene un numero con una "C", al principio ejemplo C489449, en la siguiente celda se verifica y guarda alguna relación con los valores negativos en "quantity"

```
order_canceled = df_full[df_full['invoice'].astype(str).str.startswith('C')]
display(order_canceled.head(10))
order_canceled.shape[0]
```

Aquí filtramos en la columna "invoice" los valores que tienen string en este caso una "C" con el método `startswith()` se utiliza para verificar si una cadena de texto (string) comienza con un prefijo específico, devolviendo True si es así y False en caso contrario. Pero primero se usa `.astype()` como ya mencione para pasar la columna del dataframe a cadena de texto para poder usar `startswith()`

Se observa que los productos devueltos son denominadas como valores negativos en "Quantity" y se le agrega o se asigna una C a su codigo en "Invoice" debido que la cantidad de filas (18.744) y los 10 primeros valores no cambia es decir son los mismos datos

```
#Se define un dataframe para un posterior estudio de productos cancelados
df_cancelled = order_canceled.copy()
print(f"Cantidad de productos devueltos:
\033[1m{df_cancelled.shape[0]}\033[0m")
```

En esta celda se define el dataframe que conservara los productos devueltos o valores negativos y se imprime nuevamente la cantidad, la creación de este dataframe es con la finalidad de analizar estos productos devueltos

```
#Se define un dataframe con valores positivos para el estudio de RFM
df_rfm = df_full[df_full['quantity'] > 0].copy()
print(f"Cantidad de productos vendidos: \033[1m{df_rfm.shape[0]}\033[0m")
```

Esta es la ultima celda de la limpieza de estos datos, en esta se crea el dataframe que se usara para el análisis del RFM y se estaría usando 805620 valores todos positivos, sin NaN, y sin otros elementos que pudiera dar valores confusos o erróneos para los análisis posteriores, es cierto que se estaría trabajando con apenas el 78% de los datos originales pero así es la limpieza de datos ya que se desea datos limpios y de calidad.

Continua en la segunda parte de esta entrega…..